Storage application performance boost with zero thrashing network stack

Or Gerlitz (Mellanox)

Motivation & where are we aiming

- The overhead related to copy is notable for nvme tcp workloads
- This has larger impact on SmartNIC / SoC systems which have more limited caching and memory systems
- apply Linux zero copy socket APIs to optimize these workloads in SmartNIC NVME emulation environment

application - stack/driver models for TCP zero copy

- TX
  - application buffer is pinned for the xmit + ack duration
  - buffer reclaim notification provided to the application
  - does not require special NIC HW assistance

- RX
  - model #1: provide the driver packet buffer to the application + way to re-cycle buffers
  - model #2: provide the app buffer to the driver for packet receiving
  - typically require NIC HW assistance
    - #1 header-data split (HDS)
    - #2 direct data placement (DDP)

Linux socket API extensions for zero copy

- TX
  - sendmsg with MSG_ZEROCOPY
  - kernel 4.14
  - by Willem de Bruijn / Google
  - netdev 2.1

- RX
  - mmap support for sockets (model #1)
  - kernel 4.18
  - by Eric Dumazet / Google
  - netdev 0x14

nvme-tcp / TCP stack / NIC driver layering options

1. all kernel space
   - nvme-tcp driver, TCP stack, NIC driver
   - using in-kernel sockets

2. combined
   - user-space nvme-tcp driver using sockets
   - running over kernel [sockets → TCP stack → NIC driver]

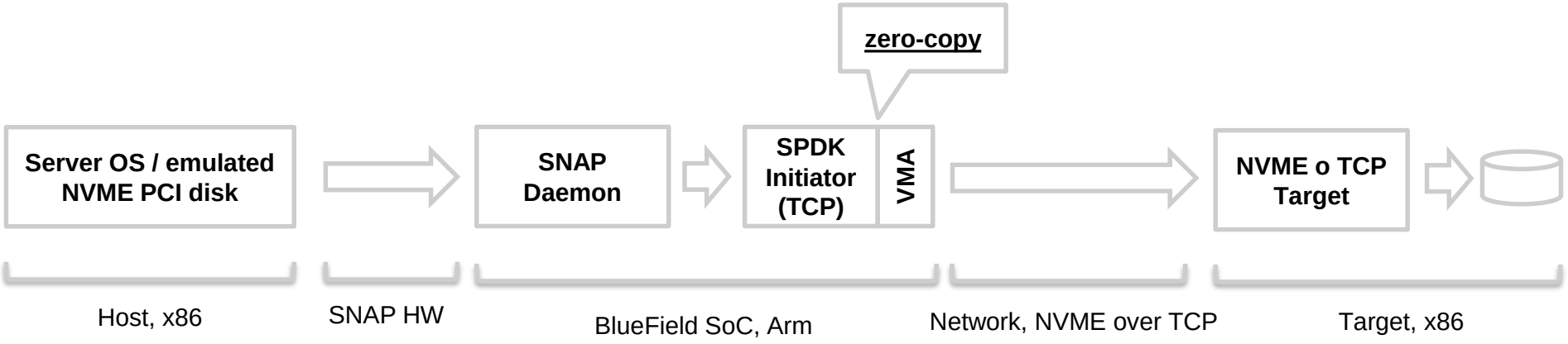
3. **all user-space**
   - nvme-tcp driver using sockets
   - TCP stack (via socket offload techniques)
   - NIC driver (part of the TCP stack or DPDK)

user-space storage and TCP stacks for this work

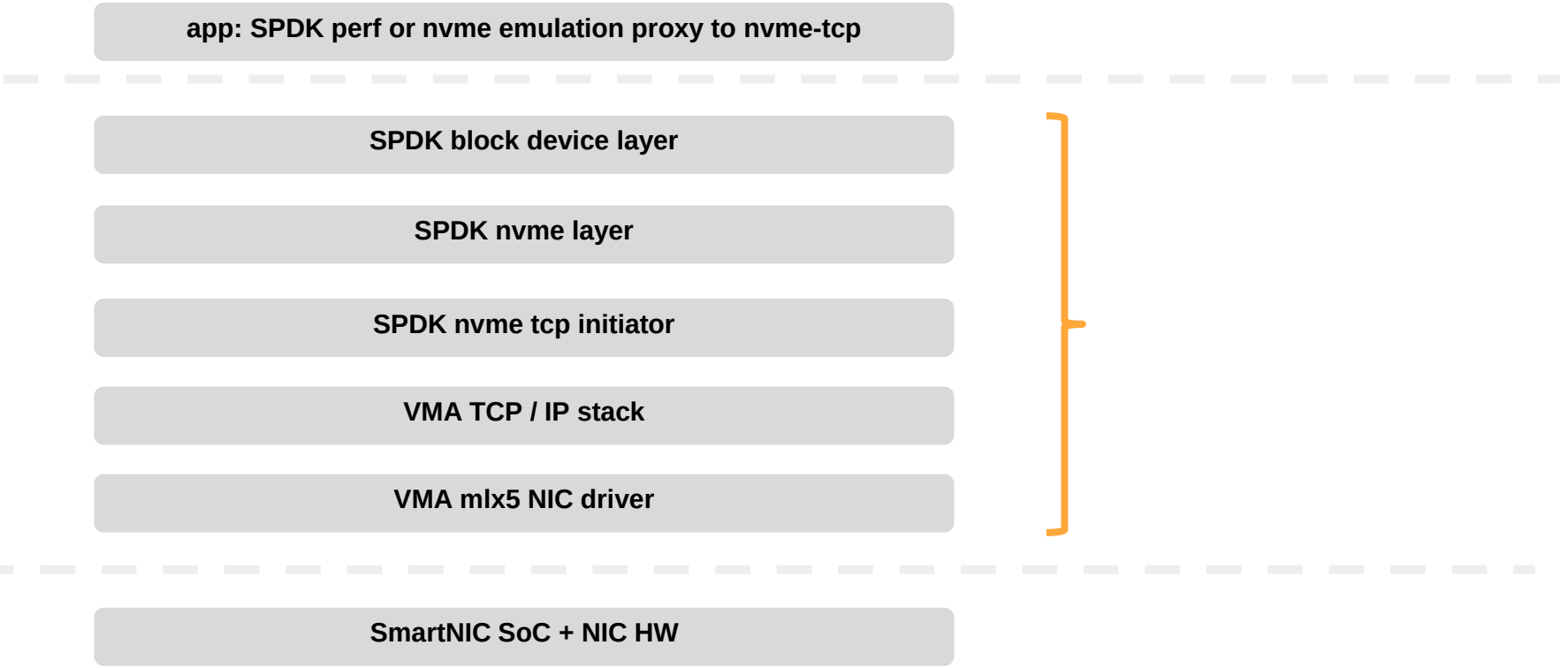
- SPDK http://www.spdk.io/
  - Storage Performance Development Kit
  - open source high performance, scalable, user-mode storage stack

- VMA https://github.com/Mellanox/libvma
  - Mellanox's Messaging Accelerator
  - open source socket offload library (using LD_PRELOAD)
  - allows non-modified applications written over socket API to run from user-space with full kernel and network stack bypass over Mellanox NIC HW

SNAP NVME emulation to NVME TCP proxy



nvme tcp initiator software layers for this work



Linux sockets TX zero copy details

- **discovery**: *getsockopt(fd, SOL_SOCKET, SO_ZEROCOPY, ..)*
- **usage declaration**: *setsockopt(fd, SOL_SOCKET, SO_ZEROCOPY, ..)*
- **transmission**: *sendmsg(fd, msghdr, MSG_ZEROCOPY)*
  - per syscall optional flag
  - *msghdr* points to an IOV which spans multiple disjoint buffers
- **buffer reclaim notification:** *recvmsg(fd, &msg, MSG_ERRQUEUE)*
  - app and stack maintain per ZC syscall invocation ID counter
  - use socket error queue to receive a [low, high] range of **acknowledged** IOV buffer IDs

adjusted elements for the ZC performance study

- ZC buffer pinning
  - **idea**: amortize the buffers pin/unpin overhead around every syscall

  - use Huge-Pages based DMA buffers for both protocol headers and data buffers
  - maintain a HW registration cache for huge-pages at the socket provider layer

- ZC buffer reclaim notifications
  - **idea**: avoid application slowdown due to waiting for ZC notifications

  - **the initiator protocol is transactional** → skip ZC buffer reclaim notifications
  - once the transaction is over, payload is clearly at the target
  - lost acks are painful, however the stack has RO access to TX buffers (huge-pages are practically pinned in Linux, so we are safe)

ZC implementation

- SPDK nvme-tcp initiator
  - use huge-pages based DMA memory for both protocol headers and data buffer
  - use MSG_ZEROCOPY flag for sndmsg call

- VMA library
  - misc changes in all layers: socket, TCP, IP and NIC driver to support both nZC and ZC socket/packets buffers/descriptors in multiple flows
  - huge-page based registration cache for ZC buffers

ZC testing setup

- (proxy) initiator SW
  - SPDK 19.07 + ZC patches
  - VMA 8.9.2 + ZC patches
  - OS: Centos 7.6 with kernel 4.14

- (proxy) initiator HW
  - 100Gbs **BlueField-1**
  - Arm A72 16 cores @ 800Mhz
  - NIC FW 18.25.6000

- Target details
  - SW: SPDK 19.07 nvme-tcp target over kernel sockets with null backing-store
  - OS: RHEL 7.6 kernel 3.10.0-693.el7
  - HW: Intel X86_64 CPU E5-2697A v4 @ 2.60GHz / 100Gbs ConnectX-5

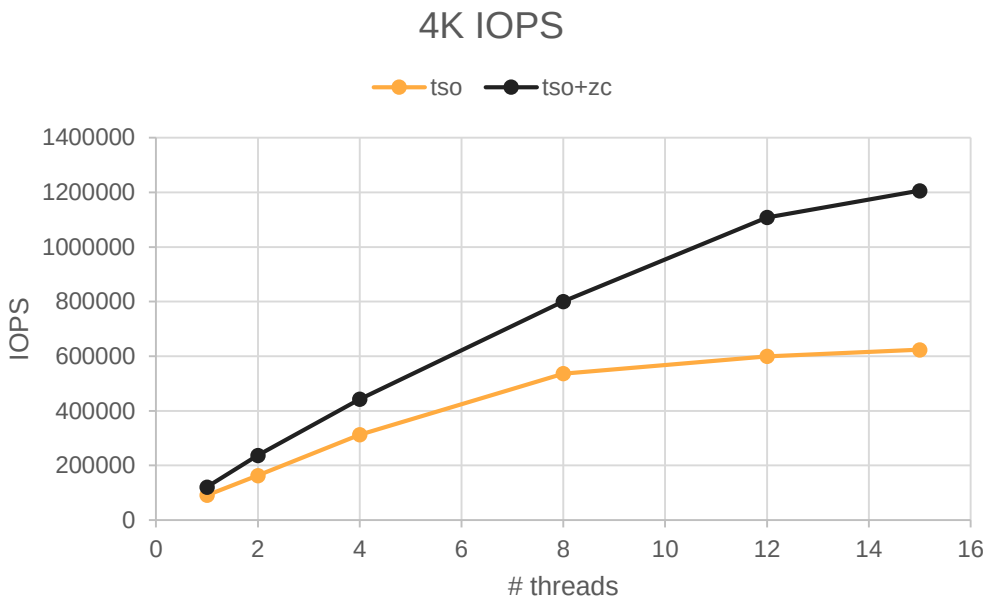
ZC testing workloads and raw results

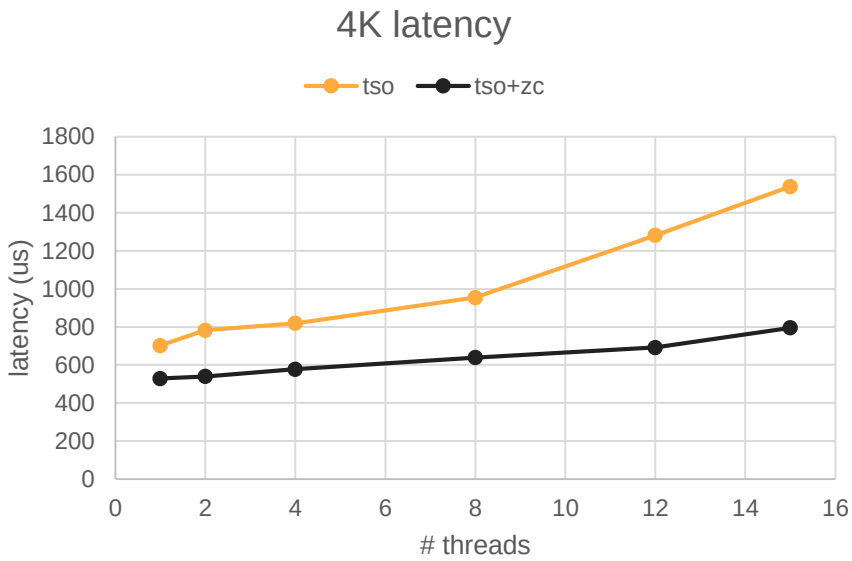
- benchmark tool and params
  - spdk perf from SoC, IO rand-write, queue-depth 64
- benchmark workloads
  - test1: varying number of cores (= threads/connection) - 1/2/4/8/12/15
  - test2: varying IO size – 512B / 1K / 2K / 4K

- benchmark results for 4KB IO size
  - one core: nZC – 85K IOPS / ZC – 120K IOPS
  - 15 cores: nZC – 650K IOPS / ZC – 1.2M IOPS
  - note: FWS (Full Wire Speed) for 4KB over 100Gbs link is 2.7M IOPS

- results for NVME emulation proxy to nvme-tcp with 4KB IO size
  - run spdk perf from the host over the emulated disk
  - nZC – 500K IOPS / ZC – **1M IOPS**

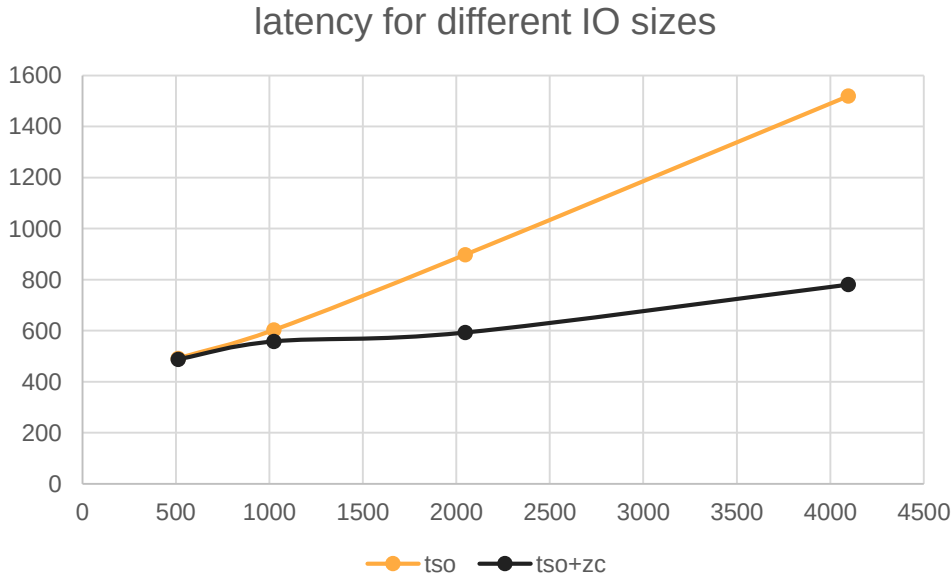
4K IO with varying number of threads (cores) - QD=64





15 cores with varying IO size: 512, 1K, 2K, 4K – QD=64





non ZC one vs. 15 cores – notable copy cost

```
14.01%  libc-2.17.so          [.] _wordcopy_fwd_dest_aligned
 8.22%  libc-2.17.so          [.] _wordcopy_fwd_aligned
 5.50%  libpthread-2.17.so    [.] pthread_spin_lock
 4.97%  libvma.so.8.9.2       [.] tcp_write
 2.63%  libvma.so.8.9.2       [.] sockinfo_tcp::rx
 2.54%  libpthread-2.17.so    [.] pthread_spin_unlock
 2.53%  perf                  [.] nvme_tcp_qpair_process_send_queue
 2.20%  libc-2.17.so          [.] memcpy
==============================================================================
16.17%  libc-2.17.so          [.] _wordcopy_fwd_dest_aligned
13.39%  libc-2.17.so          [.] _wordcopy_fwd_aligned
 5.89%  libvma.so.8.9.2       [.] tcp_write
 3.83%  libpthread-2.17.so    [.] pthread_spin_unlock
 3.65%  libpthread-2.17.so    [.] pthread_spin_lock
 3.28%  libc-2.17.so          [.] memcpy
 2.72%  libvma.so.8.9.2       [.] tcp_output
 2.35%  libvma.so.8.9.2       [.] tcp_tx_pbuf_alloc
```

ZC one vs 15 cores, copy cost eliminated

```
  8.07%  libpthread-2.17.so    [.] pthread_spin_lock
  4.61%  libvma.so.8.9.2       [.] tcp_write
  4.51%  libvma.so.8.9.2       [.] sockinfo_tcp::rx
  4.17%  libpthread-2.17.so    [.] pthread_spin_unlock
  3.12%  libvma.so.8.9.2       [.] pbuf_cat
  2.74%  libvma.so.8.9.2       [.] ring_simple::mem_buf_tx_get
  2.50%  libvma.so.8.9.2       [.] tcp_output
  2.45%  libvma.so.8.9.2       [.] cq_mgr_mlx5::poll
==============================================================================
  8.66%  libpthread-2.17.so    [.] pthread_spin_lock
  6.80%  libvma.so.8.9.2       [.] tcp_tx_pbuf_alloc
  6.41%  libvma.so.8.9.2       [.] tcp_output
  5.90%  libvma.so.8.9.2       [.] tcp_tx_seg_free
  5.18%  libvma.so.8.9.2       [.] tcp_write
  4.55%  libpthread-2.17.so    [.] pthread_spin_unlock
  3.37%  libvma.so.8.9.2       [.] ring_simple::mem_buf_tx_get
  2.32%  libvma.so.8.9.2       [.] pbuf_cat
```

What makes zero copy much more efficient?

- let us look into various low-level statistics
  - L1 cache hits (local variables) – irrelevant
  - L2 cache hits (SW contexts)
  - L3 (LLC) cache hits (**data**)
  - **IPC** – Instructions Per Cycle ← **typically good indication for efficiency**
  - Instructions per IO
  - Cycles per IO

looking into why copy cost gets higher

method	cores	IOPS	IPC	l2c-hits	llc-hits	cyc-IO	inst-IO
nZC	1	95K	1.03	0.95	0.86	8300	8600
ZC	1	130K	1.09	0.97	0.93	6100	6700
nZC	15	680K	0.53	0.81	0.60	17000	9000
ZC	15	1.4M	0.80	0.93	0.90	8400	6800

- in the nZC scheme two buffers are pulled into the SoC LLC
- many LLC cache misses → IPC decrease → IOPS not scaling
- using copy has notable system wise cost

TX zero copy @ open source / some further work

- TX ZC - open source state of the nation
  - merged to Google RPC
  - merged to SPDK version 20.01 for the nvme-tcp target side, provided nice performance boost

- investigate ways to allow apps avoid per IO data buffer pin/unpin (io_uring)
  - avoid get/put_user_pages per every ZC syscall X IOV buffer

- extend *sendmmsg* to support ZC with IOV with ZC hint per buffer
  - allow more flexible ZC scheme for apps, e.g. avoid ZC on protocol headers

- see if/where we can relax / refine ZC buffer reclaim notifications overhead
  - consider ignoring notifications for transactional cases

Takeaways

- SoCs may be sensitive to LLC cache misses
- when scaling, the improvement in IOPS for ZC goes way beyond the CPU cycles %% seen for copy through flame-graphs
- assuming large symmetry between RX and TX flows, RX zero-copy is expected to provide nice boost for similar storage workloads
- zero thrashing network stack has critical role for SmartNIC SoC